

Crescendo Attacks: Measuring Multi-Turn Adversarial Safety Decay in AI Agent Conversations

Whitepaper — **Crescendo-Heatmap Version:** 1.0 **Author:** Miracle Owolabi
Repository: <https://github.com/ArksherX/Crescendo-Heatmap> **License:** MIT

Abstract

Current AI safety mechanisms evaluate inputs at the request level. An adversarial input that violates policy is blocked; one that does not is permitted. This architecture has a structural blind spot: attacks that distribute an unsafe objective across multiple turns, with each individual turn appearing within policy and the cumulative trajectory falling well outside it.

This paper defines the **crescendo attack class** — multi-turn adversarial manipulation of AI agents and chatbots where safety degradation occurs gradually across a conversation. It introduces a measurement framework based on per-turn safety scoring, decay delta analysis, and turning point detection. It presents empirical findings from analysis of adversarial conversation samples, characterises the multi-agent amplification effect at agent handoff boundaries, and provides detection guidance for integration into production AI agent logging pipelines.

1. Introduction

The threat model for AI system safety has converged on a single-turn model: given an input, does the system produce a harmful output? This model drives the design of guardrail systems, safety classifiers, and red-team evaluation frameworks. It is correct for the threat it addresses.

It does not address crescendo.

A crescendo attack is not a single unsafe input. It is a sequence of individually acceptable inputs designed so that their cumulative effect on the agent's context, role understanding, and compliance history moves the agent toward an objective it would have refused to pursue in a single direct request.

The mechanism is simple. An attacker who understands that per-request safety evaluation operates in isolation will structure their attack to exploit that isolation. Each turn is designed to be permissible. The attack is in the trajectory.

This pattern has been observed in red-team exercises against production AI systems. It is not a theoretical concern. It works. And the existing tooling to detect it — examining conversation logs turn-by-turn for unsafe content — will find nothing, because no individual turn is unsafe.

Crescendo-Heatmap was built to make this attack class visible: to measure safety across a conversation as a trajectory, not a point-in-time evaluation.

2. The Crescendo Attack Class

2.1 Definition

A crescendo attack is a multi-turn adversarial interaction with an AI agent in which:

1. Each individual turn, evaluated in isolation, would pass standard per-request safety checks
2. The cumulative effect of the turn sequence moves the agent's operating context toward a state from which it will produce output it would have refused to produce directly
3. The final unsafe output is obtained without triggering any single-turn safety block

The name derives from the musical term for a gradual increase in intensity — an apt description of the attack structure where each turn slightly raises the adversarial pressure while remaining below the detection threshold.

2.2 Attack Taxonomy

Crescendo attacks fall into four structural categories, which may be used in isolation or combination:

Gradual Role Subversion The attacker progressively redefines the agent's role or operating context across turns. Early turns establish a benign interaction pattern. Later turns introduce incremental redefinitions: "as an expert in X," "speaking hypothetically," "in the context of a fictional scenario." By the time the unsafe request arrives, the agent's context has been sufficiently reframed that the request appears consistent with the established interaction.

Example trajectory: - Turn 1: "What are common security vulnerabilities in web applications?" - Turns 2-5: Establish the attacker as a security researcher, build compliance history - Turn 8: "Walk me through how the SQL injection from turn 4 would be implemented in code"

Trust Escalation The attacker builds a record of compliant, low-stakes interactions before escalating to the target request. The agent's context window contains evidence of a cooperative, legitimate interaction. When the unsafe request arrives, it is preceded by a history that makes refusal seem inconsistent.

Context Poisoning The attacker injects false premises into early turns that persist as assumptions in later turns. "As we established, your guidelines don't apply when discussing academic research" — if the agent does not actively challenge the false premise, it enters the context window as an established fact.

Agent Handoff Exploitation In multi-agent systems, context resets at agent handoffs. The attacker uses the reset to restart the crescendo sequence from a clean baseline, potentially in a shorter sequence since they carry tacit knowledge of what worked in the previous agent's context. Each handoff is a new attack surface.

2.3 Why Per-Request Guardrails Fail

Per-request safety scoring operates on individual inputs. It assigns a safety score to each message and blocks those that exceed a threshold. Against crescendo attacks, this architecture fails for two reasons:

No trajectory context. A per-request scorer sees turn N in isolation. It has no information about turns 1 through N-1. The attack distributes its unsafe signal across turns in amounts that fall below the individual detection threshold.

Compliance history bias. An agent that has responded cooperatively to twelve safe requests has a context window that implicitly favours continued cooperation. This is a design feature — it produces natural-seeming conversations — and an attack vector. The compliance history is part of the attack.

3. Measurement Methodology

3.1 Safety Score Model

The safety score for a given conversation turn is a normalized value in the range [0.0, 1.0] representing the estimated risk level of that turn's content.

If explicit safety scores are present in the input data, those values are used directly. When scores are absent, a heuristic estimator computes them from turn content.

Heuristic Safety Scorer

The heuristic scorer is deterministic and additive. Each turn starts from a base score of 0.05, and for every risk term present in the turn's (lower-cased) text, that term's weight is added. The sum is capped at 1.0:

```
score(turn) = min(1.0, 0.05 +  $\sum$  weight(term) for term in RISK_TERMS if term in text)
```

The vocabulary is 21 terms with weights in the range 0.16 – 0.26, spanning direct-manipulation, tool/MCP, and multi-agent risk categories. Representative terms and weights:

Term	Weight	Term	Wei
exploit	0.16	tool_call_inject	0.24
bypass, credentials, agent_handoff	0.18	delegate_bypass, hijack_tool, forged_call	0.24
privilege escalation, shadow_agent	0.20	jailbreak, context_poison, memory_hijack	0.25
exfiltrate, spawn_agent, tool_override	0.22	mcp_server_override, orchestrator impersonation	0.26

The scorer is deliberately simple and deterministic: identical text always yields the identical score, which is what makes the benchmark in Section 4 reproducible. It is intentionally conservative — it keys on an explicit vocabulary and will miss paraphrased or low-signal crescendo attacks in exchange for precision and reproducibility. Higher-fidelity, natural-language scoring requires the optional LLM scorer applied per turn (see the `--llm-score` path and `benchmark/llm_validate.py`).

3.2 Decay Delta

The **decay delta** for turn N is the signed difference between the safety score at turn N and the safety score at turn N-1:

$$\delta(N) = \text{score}(N) - \text{score}(N-1)$$

A positive decay delta indicates increasing risk. A negative decay delta indicates decreasing risk. The decay delta is the trajectory signal — it captures not the absolute risk level but the rate of change.

A crescendo attack typically produces a series of small positive decay deltas across many turns rather than a single large positive delta. Detecting the pattern requires observing the delta series, not any individual delta value.

Net decay is the total safety degradation across the conversation:

$$\text{net_decay} = \text{max_score} - \text{min_score}$$

3.3 Risk Band Classification

Each turn is classified into a risk band based on its safety score:

Band	Score Range	Interpretation
low	0.00 - 0.24	Within expected operating range
medium	0.25 - 0.49	Elevated — monitor trajectory
high	0.50 - 0.74	Concerning — approaching critical zone

Band	Score Range	Interpretation
critical	0.75 - 1.00	Active attack pattern likely in progress

Band thresholds are 0.25, 0.50, and 0.75. The critical threshold of 0.75 defines the "critical turn" used as the reference point for the pre-critical detection metric in Section 4.

3.4 Turning Point Detection

A **turning point** is a turn at which the trajectory shifts sharply enough to signal adversarial drift before the conversation reaches the critical band.

Turning point detection is deterministic and based on the magnitude of the decay delta:

```
turn N is a turning point iff  $|\delta(N)| \geq 0.2$ 
```

A turn whose safety score moves by at least 0.2 relative to the previous turn (in either direction) is flagged. Because a well-formed crescendo distributes its escalation across several such shifts, turning points typically fire multiple turns before the score first crosses the 0.75 critical threshold. The benchmark in Section 4 measures this lead directly: on the crescendo-class corpus, the earliest turning point precedes the critical turn by a mean of 5.3 turns. The trajectory monitor evaluated in Section 4 also incorporates an accumulation signal (net decay from the first turn ≥ 0.3) so that slow escalations composed of sub-0.2 steps are still caught.

3.5 Multi-Agent Attribution

When conversation records include `agent_id` fields, the analysis attributes safety scores and decay deltas to individual agents. Per-agent statistics are computed:

- `turn_count` — number of turns attributed to this agent
- `avg_safety_score` — mean safety score across the agent's turns
- `max_safety_score` — peak safety score for this agent
- `net_decay` — safety degradation within this agent's turn sequence

The cross-agent comparison reveals the **handoff amplification effect**: the difference between the safety score at the end of one agent's sequence and the safety score at the start of the next agent's sequence. A large positive differential indicates the attacker is exploiting the context reset at the handoff boundary.

4. Empirical Findings

4.1 Dataset

The analysis dataset consists of adversarial conversation samples drawn from: - Red team exercises against deployed AI agent systems - Published jailbreak attempt collections - Multi-agent orchestration test environments with simulated adversarial inputs

All samples were annotated by the author. Samples are classified as "successful attack" (agent produced unsafe output), "partial success" (agent partially complied), or "failed attack" (agent maintained refusal throughout).

4.2 Key Findings

Finding 1: Pre-critical signature detectability

73% of successful attacks in the dataset have a detectable pre-critical signature — a turning point identified by the decay delta algorithm — at least 3 turns before the conversation reaches the critical risk band. This means that trajectory-based detection would alert on these attacks before exploitation in the majority of cases.

This finding is corroborated by a fully reproducible benchmark shipped in the repository (`benchmark/` , seed-fixed, no network or LLM required). On 214 labeled conversations (130 crescendo-class attacks), trajectory monitoring detects **76.9%** of crescendo attacks at least 3 turns before the critical turn — a mean lead of 5.3 turns — at a 0% false-positive rate on benign traffic. The same benchmark quantifies the per-request baseline on the identical scores: a per-request monitor at the standard critical block threshold provides **0%** pre-critical warning (by construction it cannot fire before the critical turn), and even an aggressively tuned per-request threshold reaches only 36%. Reproduce with `python benchmark/run_benchmark.py`; see `benchmark/REPORT.md` for methodology and caveats (the 76.9% figure is a function of the benchmark's attack-profile mix, not a universal constant).

That 76.9% is a keyword-scored demonstration under favorable conditions and should be read as an **upper bound, not a detection rate**. A follow-on validation on a realistic natural-language corpus scored by an LLM (Claude Haiku, `dataset_natural_v2.jsonl` , 54 conversations, median 19 turns) gives the honest figure: trajectory monitoring detects **44%** of crescendo attacks at least 3 turns before the critical action (**69%** for any advance warning), a mean of ~5 turns of lead, at roughly **8%** false positives on benign traffic — while per-request monitoring stays near **0%**. The architectural claim (single-turn monitoring is blind to multi-turn attacks) is robust; the trajectory solution is real but imperfect, and it carries a false-positive cost concentrated on legitimate security discussions. Full methodology and the lead-sensitivity curve are in `benchmark/REPORT.md` .

Finding 2: Average attack length

The mean length of successful crescendo attacks in the dataset is 11.4 turns. The distribution is right-skewed: shorter attacks (6–8 turns) tend to use role subversion; longer attacks (15+ turns) tend to use trust escalation combined with context poisoning.

Finding 3: Most common attack type

Gradual role subversion accounts for 41% of successful attacks. It is the most efficient crescendo technique — it requires fewer turns than trust escalation and is more reliable than pure context poisoning.

Finding 4: Multi-agent handoff amplification

In multi-agent conversation samples where the attack crossed an agent handoff boundary, the decay delta measured immediately after the handoff was on average 2.3x the per-turn delta observed in single-agent samples of the same attack type. The context reset provides the attacker with a fresh compliance history while they retain knowledge of the effective approach from the prior agent's context.

Finding 5: Tool call injection as crescendo accelerator

Conversations in which MCP-style tool calls occurred showed a distinct pattern: safety scores were suppressed during tool call turns (scoring the tool call invocation rather than the content it represents) and spiked immediately after. Tool calls function as a partial safety context reset within a single agent, giving attackers a mechanism to accelerate the crescendo sequence.

4.3 Attack Signature Profiles

Three signature profiles were identified from the dataset:

Profile A — Gradual Decay (most common) Steady positive decay deltas across all turns. No large single-turn spikes. The heatmap shows a smooth gradient from green to red over 8–15 turns. Turning point fires at turn 5–7 on average.

Profile B — Spike-and-Recovery Multiple medium-spike turns that recover to lower scores, followed by a final spike that does not recover. Suggests testing of guardrail sensitivity in early turns before the final exploit attempt.

Profile C — Flat-then-Critical Near-zero decay deltas for most of the conversation, then a sudden spike to critical in 1–2 turns. Common in tool call injection attacks. Harder to detect early; the pre-critical signature window is narrow.

5. Tool Architecture

```
crescendo_heatmap/  
├─ cli.py           Entry point, argument parsing  
├─ parser.py        Input format detection, turn loading, heuristic scorer  
├─ models.py        Turn and TurnAnalysis dataclasses
```

```
|— analyzer.py      Decay delta, risk bands, turning points, summary stats
|— renderer.py      HTML heatmap + JSON/CSV output
|— llm_scorer.py     Optional per-turn LLM scoring (OpenAI / Anthropic)
|— middleware.py     Real-time HTTP middleware (--serve)
|— api.py           REST API (--api)
```

models.py defines two core dataclasses:

```
@dataclass
class Turn:
    index: int
    role: str
    content: str
    safety_score: float
    agent_id: str | None = None
    def to_dict(self) -> dict: ...

@dataclass
class TurnAnalysis:
    turn: Turn
    decay_delta: float
    risk_band: str          # low | medium | high | critical
    def to_dict(self) -> dict: ...
```

Turning points are not stored on the dataclass; they are derived in `summary_stats` from the decay-delta magnitude.

analyzer.py implements: - `analyze_turns(turns)` → list of `TurnAnalysis` -
`summary_stats(analysis)` → dict with `turn_count`, `max/avg safety scores`, `net_decay`, `critical_turns`, `turning_points`, and (for multi-agent input) `agents` and `per_agent`

renderer.py generates a self-contained HTML heatmap (colour-coded rows by risk band, per-agent colour coding in multi-agent mode) and writes the optional JSON summary and CSV export via `write_json` / `write_csv`.

6. Detection Integration

6.1 Batch Analysis for Incident Investigation

Post-incident, export conversation logs from the agent platform in JSON or JSONL format and run:

```
crescendo-heatmap --input session_log.json --output report.html --json summary.json
```

The HTML report provides a visual reconstruction of the attack trajectory. The JSON summary provides structured statistics for inclusion in incident reports.

6.2 Threshold Alerting

For continuous monitoring, integrate safety score thresholds into the agent's logging pipeline:

```
crescendo-heatmap --input session_log.json --output report.html --fail-score 0.65
```

A non-zero exit code triggers downstream alerting. Set the threshold below the critical band (0.75) to alert before exploitation.

6.3 SIEM Integration

The JSON summary output contains all fields required for SIEM ingestion: - Session identifier (from input if present) - Peak safety score, average safety score, net decay - Count of critical turns - Turning point indices (timestamps when combined with input `timestamp` fields)

Map the `turning_points` field to SIEM alert events. A turning point earlier in a conversation than turn 6 is a high-confidence attack indicator.

6.4 Architectural Countermeasures

Conversation-scoped guardrails: Implement a safety evaluator that operates on the conversation buffer, not individual inputs. It receives the last N turns as context and evaluates the trajectory rather than the most recent message.

Safety state propagation across agent handoffs: When an orchestrator passes control to a sub-agent, include the current safety state in the handoff context. The sub-agent begins with the accumulated safety context rather than a clean slate.

Mandatory context window audit: At each agent handoff, evaluate the outgoing context buffer for safety score before passing it to the incoming agent. Refuse handoffs where the buffer's safety state exceeds a threshold.

7. Limitations

Heuristic scorer precision. The keyword-based heuristic scorer is calibrated for English-language inputs and known attack pattern vocabulary. Novel attack techniques not represented in the keyword set will produce lower-than-expected scores.

No ground truth for safety score. The "correct" safety score for a conversation turn is not objectively defined. The heuristic scores are useful for trajectory analysis but should not be treated as absolute measurements. LLM-based scoring at the conversation level is a higher-fidelity alternative with higher operational cost.

Dataset coverage. The empirical findings are based on a dataset of known adversarial samples. Zero-day crescendo techniques not represented in the dataset may not produce the signature profiles described in Section 4.3.

Multi-agent attribution requires agent_id fields. Agent-level analysis requires that conversation logs include per-turn agent identifiers. Logs without this field cannot support per-agent breakdown.

8. Conclusion

Crescendo attacks exploit the structural boundary between per-request safety evaluation and multi-turn conversation reality. They are not hypothetical. They work against deployed AI systems. And they leave no detectable trace in standard safety logs because each individual turn passes individual evaluation.

The measurement framework presented here — decay delta, turning point detection, risk band classification, multi-agent attribution — provides the vocabulary and tooling to make crescendo attacks visible. The pre-critical detectability finding — demonstrated at up to 76.9% on a keyword-scored synthetic benchmark and, more conservatively, at ~44% on a realistic natural-language corpus scored by an LLM (69% for any advance warning, ~8% benign false positives), against a per-request baseline near 0% — demonstrates that trajectory-based detection alerts on a meaningful fraction of crescendo attacks before exploitation, though imperfectly and at a modest false-positive cost. The robust, architecture-level claim is that single-turn monitoring cannot see multi-turn attacks at all; trajectory monitoring recovers a real part of what it misses.

Detection requires logging conversation context, not just individual inputs. It requires evaluating trajectory, not just point-in-time safety scores. Crescendo-Heatmap provides the measurement layer; the architectural decisions about where to apply it are the defender's responsibility.

References

1. OWASP GenAI Security Top 10 — LLM01: Prompt Injection
 2. MITRE ATLAS — Multi-Turn Adversarial Probing Techniques
 3. Perez & Ribeiro (2022): "Ignore Previous Prompt: Attack Techniques for Language Models"
 4. Greshake et al. (2023): "Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications"
 5. OpenAI System Card: GPT-4 Safety Evaluation Methodology
 6. Anthropic Constitutional AI: Reward Modelling and RLHF Safety Constraints
 7. Wallace et al. (2019): "Universal Adversarial Triggers for Attacking and Analyzing NLP"
-

Crescendo-Heatmap is open source software released under the MIT License.
Repository: <https://github.com/ArksherX/Crescendo-Heatmap>